

TriageDesk · informe de P6

Borrador para integrar al informe del equipo · 4 de octubre de 2026. Las numeraciones siguen CONTEXTO_PROYECTO.md; el índice académico final no estaba disponible y debe confirmarse antes de integrar estas secciones.

Integrantes

- Francisco Nappa (P6; autor de commits: Francisco Eduardo Nappa).
- Ana Fare.
- Florencia Quiñonez.
- Juan Lopez Korach.
- Santiago Gil Ishikawa.
- Gustavo Forcherio.

Las asignaciones P1-P5 deben ser confirmadas por el equipo.

1. Resumen del proyecto

TriageDesk es una mesa de ayuda donde el solicitante describe un problema en texto libre. Un modelo de lenguaje sugiere categoría, urgencia, impacto y módulo. El dominio de tickets decide prioridad y SLA con reglas deterministas. Los casos críticos se escalan y generan eventos para notificaciones y reportes.

P6 implementa reporting-service: recibe esos eventos, mantiene una vista de lectura y responde consultas de resumen, tickets y cumplimiento de SLA. Permite observar el resultado operativo sin consultar ni modificar el agregado de tickets. La demo local de P6 emite eventos de ejemplo; no certifica el flujo de creación, clasificación ni envío de notificaciones de los demás servicios.

2. Arquitectura y decisiones

La arquitectura acordada utiliza seis microservicios y bases independientes. El gateway es la entrada pública y valida JWT. Tickets y usuarios utilizan Java/PostgreSQL, la clasificación Python/MongoDB y notificaciones/reportes NestJS/MongoDB. Esta combinación está definida en el contexto del equipo.

Para la primera defensa, los eventos viajan por HTTP. reporting-service escucha en el puerto 8085 y almacena exclusivamente en reporting_db. POST /api/eventos es interno; GET /api/reportes/* se expone mediante el gateway.

En P6 el controlador valida el sobre. EventosService registra un inbox persistente, selecciona una Strategy y aplica un upsert atómico. TicketViewRepository actualiza solo snapshots con fecha igual o posterior a la vista. ReportesRepository usa agregaciones MongoDB para distribuir estados, prioridades y categorías, paginar consultas y agrupar el cumplimiento del SLA.

El índice único por eventId evita duplicados. Guardar PENDIENTE antes de proyectar y confirmar después permite reintentar ante fallos sin perder el evento por una confirmación prematura. Los índices se inicializan antes de atender solicitudes. La implementación funciona con MongoDB

standalone, como la infraestructura actual.

El sobre no contiene una secuencia ni fechaResolucion en el snapshot. Los empates de occurredAt dependen del orden de llegada. Si el cierre llega primero y la resolución después como evento viejo, su fecha permanece desconocida. El dato se excluye del porcentaje; se propone coordinar una revisión del contrato con P2/P5.

6. Seguridad y trazabilidad

El gateway autentica y autoriza consultas de reportes para AGENTE/ADMIN. El servicio interno confía en la red privada y no duplica la validación JWT. No debe publicarse POST /api/eventos por el gateway. El Compose propio publica localhost únicamente para desarrollo y demostración.

MONGO_URI se configura por entorno y es obligatoria. No se incluyen credenciales reales en código, ejemplos ni evidencias. Cada servicio utiliza su propio usuario de base de datos en el despliegue compartido. La demo temporal es independiente.

Los DTO rechazan campos adicionales, enums desconocidos, UUID inválidos y fechas fuera del formato acordado. El snapshot no incluye descripcion. Los mensajes de error internos no exponen stacktrace ni detalles de conexión. Los logs son líneas JSON con timestamp, level, service, correlationId y message.

13. Patrones aplicados

CQRS: ticket-service conserva el modelo de escritura y reporting-service mantiene su propia vista. La propagación asíncrona implica consistencia eventual: la consulta puede tardar en reflejar una operación reciente.

Strategy: hay un proyector por cada evento, a través de Proyector. Los seis comparten la transformación del snapshot, evitando recalcular prioridad o SLA. La política de resolución contempla cierre y reapertura.

Repository: TicketViewRepository, EventoProcesadoRepository y ReportesRepository encapsulan escritura condicional, inbox y agregaciones. Controladores y servicios no necesitan manejar el modelo Mongo directamente.

16. Trabajo con Git y validación

La implementación se divide en commits locales con Conventional Commits en español en feat/reporting-proyecto. Solo incluye el servicio y los entregables asignados a P6 (contrato de reportes y diagrama propio). Los cambios preexistentes en archivos compartidos quedan fuera de esos commits. No se realiza push ni se crea un PR.

La validación comprende compilación, TypeScript estricto, 22 pruebas unitarias y 39 pruebas de integración HTTP con MongoDB 7 real. Incluye concurrencia, entrega duplicada y desordenada, recuperación, reaperturas, errores comunes, paginación, health y coherencia del Swagger con OpenAPI. Una demo adicional ejecuta dist/main.js y captura respuestas reales para el anexo de evidencias.

El Dockerfile y el Compose propio están preparados. El build del contenedor queda pendiente en esta terminal porque Docker Desktop no está integrado con WSL. También quedan pendientes la integración de P1/P2, los tests del flujo completo, las revisiones del equipo, el proveedor de IA real y los ensayos cronometrados.

Anexo de evidencias

El archivo evidencias.json registra fecha, entorno, método, ruta, estado HTTP y respuesta de la demo local. La exportación PDF incluye una representación de esas respuestas, rotulada como evidencia local de P6. No reemplaza las capturas del gateway, la clasificación real ni la demo completa del equipo.

Fuentes

- CONTEXTO_PROYECTO.md y exportación local de Trello.
- NestJS: <https://docs.nestjs.com/techniques/mongodb>
- Validación NestJS: <https://docs.nestjs.com/techniques/validation>
- MongoDB updateOne:
<https://www.mongodb.com/docs/manual/reference/method/db.collection.updateOne/>

Respuestas HTTP reales · demo local P6

Captura textual obtenida en 2026-10-04T16:37:21.635Z

JavaScript compilado + proceso MongoDB real temporal standalone; sin servicios P1-P5

Health · GET /health → HTTP 200

```
{
  "status": "UP",
  "service": "reporting-service",
  "version": "0.1.0"
}
```

Creacion · POST /api/eventos → HTTP 202

```
{
  "eventId": "77dd2b8f-9356-43e0-8c4d-5f205daeacd8",
  "resultado": "PROCESADO"
}
```

Escalamiento · POST /api/eventos → HTTP 202

```
{
  "eventId": "170ebc0e-4ffb-41ae-92d0-e2a3cdc3c5d4",
  "resultado": "PROCESADO"
}
```

Duplicado · POST /api/eventos → HTTP 202

```
{
  "eventId": "170ebc0e-4ffb-41ae-92d0-e2a3cdc3c5d4",
  "resultado": "DUPLICADO"
}
```

Evento antiguo · POST /api/eventos → HTTP 202

```
{
  "eventId": "e30c13e2-de8f-4ae9-a422-4b60eeb5b79d",
}
```

```
"resultado": "PROCESADO"
}
```

Resumen del escalado · GET /api/reportes/resumen → HTTP 200

```
{
  "total": 1,
  "abiertos": 1,
  "escalados": 1,
  "slaVencidos": 1,
  "porPrioridad": {
    "P1": 1,
    "P2": 0,
    "P3": 0,
    "P4": 0,
    "SIN_PRIORIDAD": 0
  },
  "porCategoria": {
    "INCIDENTE": 1,
    "SOLICITUD": 0,
    "CONSULTA": 0,
    "BUG": 0,
    "SIN_CATEGORIA": 0
  },
  "porEstado": {
    "NUEVO": 0,
    "PENDIENTE_CLASIFICACION": 0,
    "CLASIFICADO": 0,
    "ASIGNADO": 0,
    "EN_CURSO": 0,
    "ESCALADO": 1,
    "RESUELTO": 0,
    "CERRADO": 0
  },
  "generadoEn": "2026-10-04T16:37:21.572Z"
}
```

```
}
```

SLA vencido · GET /api/reportes/sla → HTTP 200

```
{  
  "vencidos": [  
    {  
      "ticketId": "1ecab94d-d47a-48f3-a1b5-62e20a4ef64e",  
      "titulo": "Produccion caida (demo P6)",  
      "prioridad": "P1",  
      "estado": "ESCALADO",  
      "fechaLimiteSla": "2026-10-04T15:37:22.409Z",  
      "horasVencido": 1  
    }  
  ],  
  "cumplimientoPorPrioridad": {  
    "P1": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    },  
    "P2": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    },  
    "P3": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    },  
    "P4": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    }  
  }  
}
```

```
    }  
  },  
  "generadoEn": "2026-10-04T16:37:21.588Z"  
}
```

Tickets filtrados · GET

/api/reportes/tickets?prioridad=P1&estado=ESCALADO → HTTP 200

```
{  
  "content": [  
    {  
      "ticketId": "1ecab94d-d47a-48f3-a1b5-62e20a4ef64e",  
      "titulo": "Produccion caida (demo P6)",  
      "estado": "ESCALADO",  
      "prioridad": "P1",  
      "categoria": "INCIDENTE",  
      "moduloAfectado": "AUTENTICACION",  
      "escalado": true,  
      "solicitanteId": "6b32a255-f60c-40c7-8c0e-df04f7150102",  
      "agenteAsignadoId": null,  
      "fechaCreacion": "2026-10-04T14:37:21.409Z",  
      "fechaLimiteSla": "2026-10-04T15:37:22.409Z",  
      "fechaResolucion": null,  
      "ultimoEventoEn": "2026-10-04T14:37:22.409Z"  
    }  
  ],  
  "page": 0,  
  "size": 20,  
  "totalElements": 1,  
  "totalPages": 1  
}
```

Resolucion fuera de termino · POST /api/eventos → HTTP 202

```
{
```

```
"eventId": "f517b221-d342-45d7-9197-3af42cbd58fe",  
"resultado": "PROCESADO"  
}
```

Cumplimiento tras resolver · GET /api/reportes/sla → HTTP 200

```
{  
  "vencidos": [],  
  "cumplimientoPorPrioridad": {  
    "P1": {  
      "resueltos": 1,  
      "enTermino": 0,  
      "porcentaje": 0  
    },  
    "P2": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    },  
    "P3": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    },  
    "P4": {  
      "resueltos": 0,  
      "enTermino": 0,  
      "porcentaje": null  
    }  
  },  
  "generadoEn": "2026-10-04T16:37:21.625Z"  
}
```